



CamTech

— KNOWLEDGE, REASON, CHARACTER —

Cambodia University of Technology and Science

EB Reaksmey

A Graduation Project Report in Partial Fulfillment of the
Requirements for the Degree of

Bachelor's Degree in Cybersecurity

Cambodia University of Technology and Science

Faculty of Engineering

Security Implementation for Guarantee of Payment System

By

EB Reaksmeay

(ID: 6023010017)

Supervisor : Dr. Engelking Elias Jacob

Mentor : Mr. Long Makara

September, 2026

Phnom Penh, Cambodia

Declaration of Originality

I, **EB Reaksmey**, hereby declare that this Graduation Project Report, titled “**Security Implementation for Guarantee of Payment System**”, submitted in partial fulfillment of the requirements for the degree of Bachelor of **Cybersecurity Engineering** at the Cambodia University of Technology and Science, is my own original work. This work was carried out during my industrial placement at **Intercare Hospital** from **March 5th, 2026** to **September 4th, 2026**. All sources and references used were properly acknowledged and cited.

Full name : EB Reaksmey

Date : 01, September 2026

Report Approval

This report, titled “**Security Implementation for Guarantee of Payment System**”, submitted by **EB Reaksmey** (ID: **6023010017**), has been approved by the following members of the examination committee as a partial fulfillment of the requirements for the degree of **Bachelor degree in Cybersecurity Engineering** at the **Cambodia University of Technology and Science**.

Report Submission Approval

Supervisor : **Dr. Engelking Elias Jacob**

Head of Digital Health, Intercare Hospital

Mentor: **Mr. Makara Long**

Lecturer, CamTech University

Examination Committees

Committee 01: _____

Committee 02: _____

Committee 03: _____

Committee 04: _____

Committee chair: _____

Acknowledged by

Name, Title:

Date of Approval:

Acknowledgements

I would like to express my sincere gratitude to CamTech University for the opportunity to undertake this placement, and for its support throughout, from the very start of the process to its completion.

My sincere thanks go to Intercare Hospital for hosting this placement within such a professional and welcoming environment.

I wish to express my deepest gratitude to my supervisor, Dr. Elias. You placed your trust in me even though I had no real experience and lacked the skills this work demanded. You stood by me through every situation, followed up when I needed it, and listened patiently even on my hardest days, always ready to talk things through with me. You pushed me further than I ever thought I could go, and helped me grow into someone I wasn't six months ago. This time under your guidance has been truly precious to me.

I would like to convey my sincere gratitude to my mentor, Loukru Makara. Your steady direction, especially at moments when I was uncertain how to proceed, was invaluable throughout this project. I appreciate the patience and encouragement you offered at every stage.

I am sincerely grateful to Dr. Ebner, Deputy CEO of Intercare, whose contribution made this project possible to carry through in many ways.

I extend special appreciation to the IT team, Bong Udom, Bong Chantha, and Bong Savda, who formed the backbone of this project's technical work, offering ideas, consultation, and support at every step.

I extend my sincere appreciation to Mrs. Debbie, Mrs. Riza, Bong Kimheng, Dr. Denny, Dr. Justyna, and Dr. Visoth, for your support and the time you spent answering our endless questions along the way.

To my dear friends on the GOP-MVP project, Rith (Soputhearith), who led development and communication, and Chamrath, who was responsible for data organisation and the ANZER adaptation, I am grateful for your collaboration and dedication throughout this

project. This project would not have succeeded without the work each of you contributed, and I value the effort we put in together.

Lastly, I am immensely grateful to my family for your continued support throughout this placement. Your patience and encouragement, even without always knowing the details of the work, meant more than you may realise.

Finally, I extend my heartfelt thanks to everyone else who contributed to this project in ways not individually named here. Your assistance is sincerely appreciated.

Abstract

Intercare Hospital processes insurance Guarantee of Payment (GOP) requests by hand, which causes delays, errors, and inconsistency. Automating this work places patient clinical data and hospital pricing in a single system used by five staff roles with different responsibilities. This study designed, implemented, and verified the security of that system, covering identity management, access control, audit logging, and vulnerability assessment. The NIST Cybersecurity Framework 2.0 structured the work, with STRIDE used for threat modelling, the 2021 edition of the OWASP Top 10 for code review, and a separate assessment against the OWASP Application Security Verification Standard (ASVS) 4.0.3 at Level 2. Keycloak was selected over a cloud service because the hospital required hosting on its own servers. Access control combined role-based (RBAC) and attribute-based (ABAC) rules across forty-two defined actions, supported by an append-only audit log. Verification used both automated scanning and manual role-by-role testing of the running system. Static analysis with Semgrep covered 356 files and produced 15 findings. None concerned access control. Manual role-by-role testing of the same system in the same period produced 47 tracked findings, of which 23 were access-control findings. Most were fixed before deployment, and the rest were recorded as deferred decisions. The ASVS assessment separately produced 15 further findings, of which 10 were resolved. Permissions were found to depend on three factors together: the user's role, their assignment to a case, and how far that case had progressed through the workflow. These findings indicate that automated scanning alone cannot verify authorisation logic, because such rules depend on organisational policy rather than on code structure. They also show that access control in clinical workflows must depend on the state of a case, not on the user's role alone.

Keywords: access control, role-based access control (RBAC), attribute-based access control (ABAC), Keycloak, healthcare information systems, security testing

Contents

1	Introduction	12
1.1	Background of the Study	12
1.2	Company Overview	13
1.3	Project Overview	14
1.4	Research Objectives	15
1.5	Graduation Project Report Structure	15
2	Problem Statement	17
2.1	Background of the Problem	17
2.2	Impact of the Problem	19
2.3	Scope of the Problem	20
2.4	Summary of the Problem	21
3	Literature Review	22
3.1	Access Control Models: RBAC and ABAC	22
3.2	Security Standards for Health Information Systems	24
3.3	Threat Modelling Approaches	25
3.4	Automated and Manual Security Verification	26
4	Methodology	28
4.1	Research Design	28
4.2	Frameworks and Standards Applied	31
4.3	Threat Modelling	32
4.4	Tools and Technology	35

5	Implementation	38
5.1	Identity and Access Management	38
5.1.1	Sign-in and Session Handling	38
5.1.2	Account Status and AD Federation	40
5.2	Access Control Model	42
5.3	Audit Logging	45
5.4	Deployment	48
6	Result and Discussion	51
6.1	Result	51
6.1.1	How the System Was Checked	51
6.1.2	Package and Container Scanning	52
6.1.3	Checking the Project's Own Code	53
6.1.4	Manual Role-by-Role Testing	53
6.1.5	OWASP Top Ten Assessment	58
6.1.6	ASVS 4.0.3 Assessment	59
6.2	Discussion	62
6.2.1	Permission Depends on More Than the Role	62
6.2.2	Why the Scanners Found No Access-Control Problems	63
6.2.3	Layered Checks, and the Guard That Never Ran	64
6.2.4	What This Suggests for Similar Systems	65
7	Challenges, Conclusion and Future Work	67
7.1	Challenges	67
7.1.1	Technical Challenges	67
7.1.2	Stakeholder Challenges	68
7.2	Conclusion	69
7.3	Future Work	70
	References	74

List of Tables

4.1	Frameworks and standards used, and how far each was applied.	31
4.2	STRIDE categories and the defects found in each.	33
4.3	Tools used in the system and in its verification.	36
5.1	The five roles and what each is responsible for.	43
6.1	Scanning targets and findings.	52
6.2	What Semgrep found.	53
6.3	The 47 findings by type.	54
6.4	The six kinds of access-control finding.	54
6.5	Examples of work left for later, and why.	55
6.6	OWASP Top 10:2021 ratings.	58
6.7	ASVS 4.0.3 results by chapter.	60
6.8	The three factors, with an example of each.	62
7.1	Work remaining, by priority.	70

List of Figures

2.1	The GOP process on paper compared with the process in the system. Amber marks where the two differ. A patient who declines ends the case; where the insurer asks for changes, Finance revises the request and sends it again. . . .	18
4.1	The seven phases of the security work, mapped to the functions of the NIST Cybersecurity Framework 2.0. Phases 3 to 5 were revisited as checking changed earlier decisions. Phases 6 and 7 were not complete at the time of writing.	29
5.1	The sign-in path. Keycloak checks the password and returns a token carrying the user's identity and role. A token with one valid role gets a session; a token with none is refused.	39
5.2	Account status after AD federation. Access requires two conditions to hold at once, checked on every request: the account must be enabled in Keycloak, federated from AD, and enabled in the application's own record.	41
5.3	The three factors evaluated in every access decision. All three must hold; if any one fails, the action is refused.	44

Chapter 1

Introduction

This chapter introduces the setting of the project and the problem it addresses. It describes how Guarantee of Payment requests were handled at Intercare Hospital before the system was built. It then gives an overview of the hospital and the project, states the objectives of this Graduation Project Report, and outlines the structure of the chapters that follow.

1.1 Background of the Study

Hospitals and insurance companies work together through a process known as Guarantee of Payment (GOP). Before a planned admission, the hospital asks the insurer to confirm in writing that it will cover the cost of treatment, so the patient can be treated without paying up front. The request is not a single document but a package: patient details, a clinical description of the planned procedure, statements from the treating doctors, and an itemised cost estimate. Because insurers set their own forms and required fields, assembling that package is manual work that falls on hospital staff, and the effort grows with the number of insurers a hospital works with.

At Intercare Hospital in Phnom Penh, this process was carried out manually, and a single request could pass through four departments before it reached the insurer. The result was slow, inconsistent, and tiring for the people doing it. Chapter 2 describes that process in detail.

Automating this process removes much of that effort, but it also changes where the risk sits. Information that once lived on paper in a single office now sits in a database that many people can reach across the hospital network. Decisions that were previously made in person, such as who is allowed to confirm a cost or approve a clinical statement, become rules written into software. A GOP request contains both clinical information about a patient and a financial commitment on behalf of the hospital, so a mistake in those rules is not only a privacy problem but a financial one.

Security therefore could not be treated as something added after the system worked. It covers more than one question: who is allowed into the system, what each person may do once inside, what record is kept of what they did, and whether the software and servers the system runs on are themselves safe. These questions had to be answered alongside the workflow rather than after it. This Graduation Project Report describes the requirements that were identified, the security that was designed and built to meet them, and how the result was tested and revised.

1.2 Company Overview

Intercare Hospital is a private healthcare provider in Phnom Penh, Cambodia, based at the Olympia Medical Hub Building on Street 161, Veal Vong, 7 Makara. The hospital's stated aim is to become the most trusted healthcare provider in Cambodia, with values organised under a framework called I-CARE, covering innovation, compassion, accountability, responsiveness and empowerment [1].

The hospital opened in 2020 as a general practice clinic and has added services each year since: laboratory and imaging in 2021, maternity, inpatient and emergency care in 2022, and operating theatre services, physiotherapy and a visa health check centre in 2023. Paediatrics, a child wellbeing centre and a set of community clinics under the Intercare Express name followed, with a medical centre in Siem Reap planned [1].

The hospital employs more than two hundred medical professionals, Cambodian and international, representing over twenty nationalities, and treats Cambodian families, expatriates, company employees and travellers. Care is provided in partnership with more than

forty insurance providers, which makes the Guarantee of Payment process a routine part of daily work rather than an occasional task [1].

1.3 Project Overview

This project was carried out within the hospital’s Digital Health department, which is responsible for the systems that support clinical and administrative work. The aim was to replace the manual cost estimation and Guarantee of Payment process with a web-based system, so that a request could be prepared, reviewed and sent from one place instead of being assembled by hand across several departments.

The wider effort is planned in three phases. This project delivers Phase 1, built over six months as the internal foundation that later phases can extend without being rebuilt. Phase 1 covers the hospital’s own side of the workflow: creating a case from patient and encounter data, building a cost estimate, generating the GOP request and its document output, tracking its status through approval, rejection or revision, and keeping an audit trail. The system does not connect to insurer systems. Finance reviews each case and sends the request from within the system; nothing goes out automatically. The insurer’s reply arrives in a staff Outlook mailbox rather than in the system, so Finance records the answer by hand, and a case needing revision is corrected and sent again by the same path. Claims generation and submission belong to later phases. The system’s data model follows the HL7 Fast Healthcare Interoperability Resources (FHIR) standard, which shaped several of the security decisions described later. At the time of writing, some parts of this workflow were complete and working, while others were still being finished.

The system was built by a team of three students on industry placement, each from a different major: software engineering, data science and artificial intelligence, and cybersecurity. My major is cybersecurity, and I was responsible for the security of the system. That covered working out what needed to be protected, designing and building the controls to protect it, and testing whether they held. This Graduation Project Report describes that security work.

The system was built for internal use within the hospital rather than for public access, and is hosted on the hospital’s own servers rather than in the cloud. This decision

shaped much of the security design, as the responsibility for protecting the system and the environment it runs on stays with the hospital.

1.4 Research Objectives

The general objective of this work was to build security into the Guarantee of Payment system as it was developed, rather than adding it once the system was working. The specific objectives were as follows.

1. To identify what the system needs to protect, and to establish the security requirements of the workflow, including the separations between roles that already exist in hospital practice.
2. To select security standards suitable for a small internal health system that can also be extended as the system grows, and to state how far each one is applied.
3. To design and implement the security of the system: signing in, what each user may do, how their actions are recorded, and the security of the software the system runs on.
4. To evaluate the security before deployment, using manual testing, automated scanning, and a separate assessment against the OWASP Application Security Verification Standard, and to record what was found.
5. To compare what each method detected, and to discuss what that means for verifying access control in systems of this kind.

1.5 Graduation Project Report Structure

This Graduation Project Report is organised into seven chapters.

- Chapter 1 introduces the setting, the hospital, the project, and the objectives of this work.

- Chapter 2 describes the problem: how Guarantee of Payment requests were handled manually, what that cost the hospital, and which part of it this project addresses.
- Chapter 3 reviews existing work on access control models, security standards for health information systems, threat modelling, and how access control is verified.
- Chapter 4 sets out the method, including the frameworks applied and how far each one was used.
- Chapter 5 describes what was built: how users are identified, the access control model, the audit trail, and the security of the deployment.
- Chapter 6 reports what the verification produced and discusses what those results mean.
- Chapter 7 covers the difficulties encountered, the conclusions drawn, and the work that remains.

Chapter 2

Problem Statement

This chapter sets out the problem the project addresses. It describes how Guarantee of Payment requests were handled before the system was built, what that process cost the hospital, which part of it this Graduation Project Report is concerned with, and what the problem amounts to when stated in short.

2.1 Background of the Problem

A Guarantee of Payment request is assembled from parts held by different people. The patient's identity and insurance details sit with the front desk, the clinical description comes from the surgeon, a separate assessment comes from the anaesthetist, and Finance builds the itemised cost from the hospital's own pricing. Each insurer sets its own form and required fields, so the same case may be presented differently depending on who is being asked to pay.

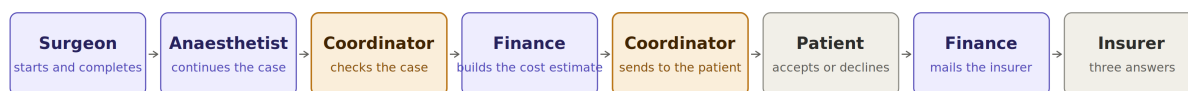
At Intercare Hospital this was done on paper, and the paper travelled. The surgeon and anaesthetist completed their parts and handed the file to the Surgical Coordinator, who checked it for missing information. Where something was missing, the Coordinator went back to the doctor concerned, who either supplied it themselves or gave permission for the Coordinator to enter it on their behalf. Once complete, the file went to Finance for costing, returned to the Coordinator for the patient's acknowledgement, and if the patient agreed went back to Finance to be sent to the insurer. If the patient declined, the case ended there.

The paper file was the control. Only the person holding it could act in that case. Nothing moved forward unless someone carried it, and an incomplete case went back to the doctor. The order of the work, the point at which each group's part ended, and the separation between the clinical and financial sides were all carried by the movement of that paper. None of it was written down as a rule, because none of it needed to be.

Automating this is not just moving the same process onto a screen. Once the file becomes a database record, everyone can reach it at any time, and the sequence the paper enforced no longer exists.

The new system removes the Coordinator as a required stop. The doctors now pass the case to Finance directly. The Coordinator now only watches progress; editing another role's part of a case is no longer something the Coordinator does. This was not the original plan. Early in the work, the Coordinator kept a narrow edit right carried over directly from paper: the ability to adjust a case's pricing after Finance had verified it, on the understanding that the change had been agreed by phone or in person first, with only the audit log to show who made it. The system had no way to check that the conversation actually happened, so the edit was allowed and simply recorded. That exception was closed before the work finished. The Coordinator's editing rights were narrowed to their own scope only, case priority and response-time targets, and the trust-based allowance was removed rather than kept.

Version 1 — on paper, Coordinator takes action



Version 2 — in the system, Coordinator steps out of the flow

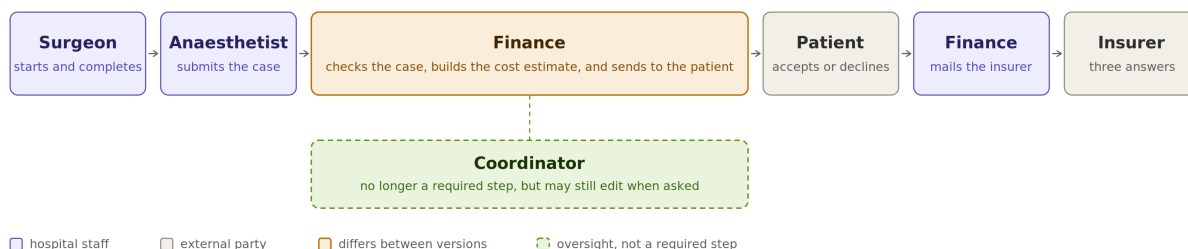


Figure 2.1: The GOP process on paper compared with the process in the system. Amber marks where the two differ. A patient who declines ends the case; where the insurer asks for changes, Finance revises the request and sends it again.

On paper, the Coordinator had to walk over and ask before changing a doctor's form. In software, nothing physical stops anyone. So each rule the paper enforced had to become one of two things. Most rules the code blocks outright: the surgeon and anaesthetist can no longer edit a case once it reaches Finance, and the system refuses the change. One rule briefly took the other path, allowed without a check and only logged, trusting that the real-world conversation had happened. That was the Coordinator's pricing edit described above, and it is the only example of that pattern found anywhere in the system. It did not stay that way for long: by the time this work finished, it had been closed and folded into the first kind of rule instead, so every rule the paper enforced is now enforced by code, with no trust-based exception left standing.

2.2 Impact of the Problem

The manual process cost staff time. Preparing one request meant collecting information that already existed elsewhere. Staff walked between departments to get it, then typed the same patient details more than once as the file moved. A request passed through four departments before it left the hospital, and each hand-off waited on someone being free to receive it.

The clearest case was the handover to Finance. The surgeon and anaesthetist wrote their parts by hand, and Finance then re-entered all of it digitally, because both the patient consent form and the insurer's request had to be produced as documents. With each insurer setting its own format, the same handwritten case could be typed out in more than one layout.

The process also depended on people who are hard to reach. Finance and the Coordinator could not move a case forward until the surgeon and anaesthetist had entered their parts, and staff report that those parts were sometimes forgotten or supplied late. Chasing a doctor who is in theatre or between patients is not quick, so a case could sit waiting on one missing figure.

Order was also difficult to keep. With cases held as paper files, there was nothing to show which had been waiting longest or which needed attention first. Staff report that a case opened early could finish last, and that an emergency case could be delayed simply

because the file was hard to find among the others. Nothing about a stack of paper tells you what is urgent.

Repetition also costs accuracy. Details typed a second or third time can be typed differently, a date entered as the fourth month rather than the third, for instance. The information the doctors supplied was correct; the copying was where it went wrong. A mismatch between the patient record, the clinical description and the cost is the kind of error an insurer returns rather than resolves. And because each insurer sets its own form, a missing field found late meant the request came back and the work was done again.

The paper carried its own record. The file showed who the doctors were, who handed it on, and when. That record existed as a by-product of being paper: a signature had to be written, a date entered, and a correction crossed out in view of whoever held the file next. Software keeps none of that by itself. A database shows the current value of a field, not who set it or what it was before. The record has to be built deliberately, or there is none.

2.3 Scope of the Problem

Not all of the problems described above are addressed here. The security work covers signing in, what each user may do, how their actions are recorded, and the security of the software the system runs on. It also covers the administration screens themselves: the interface for enabling and disabling accounts, and the view onto the audit trail. The application logic, the integration with the hospital information system, and the database design were the work of my teammates, and appear in this Graduation Project Report only where a security decision depends on them.

The system is only reachable inside the hospital, so the realistic risk is a member of staff doing something they should not, rather than an outside attacker. Two checks were outstanding at the time this scope was set: a penetration test by the hospital's IT Security Division, and a review of the container settings against the Center for Internet Security (CIS) Docker Benchmark. The first has since been carried out, by Canadia Bank (IT security department) together with the hospital's IT Security Division. The second, the container review, has also since been carried out by Canadia Bank (IT security department), though not confirmed as following the CIS Docker Benchmark specifically. Going live also

required a signed non-disclosure agreement, hospital approval, and privacy safeguards, all agreed at the start of the project.

2.4 Summary of the Problem

The manual Guarantee of Payment process was slow, repetitive and open to error. It did keep a record of who did what, but only because the paper carried signatures, dates and visible corrections.

Automating it removes the costs. It also removes two things that came free with the paper. The first is the physical route that quietly enforced the order of the work: who could act on a case, at what point their part ended, and where the clinical side stopped and the financial side began. The second is the record itself.

Software will not make people work in order, and it will not keep a history. You have to build both. Every rule must be written into the code. Every action must be saved on purpose. On paper, someone nearby might spot a mistake. In software, no one is nearby. If a rule is wrong, nothing catches it.

The question is not whether the process can be automated. It is whether the system can be trusted to let the right people do the right things. Answering it means listing every action the system allows, deciding the rule for each one, putting the check somewhere the user cannot get past, recording what happened, and then testing that it works.

Chapter 3

Literature Review

This chapter reviews the existing work this project uses. The work covered the security of the system as a whole: signing in, what each user may do, how their actions are recorded, and the security of the software the system runs on. A systematic review of access control in electronic health record systems groups this field into identification, authentication, authorisation and accountability [2], and those four categories match the strands of work described in Chapter 4.

The review is weighted towards authorisation, because that is what most of this project's findings are about. Section 3.1 covers the models of authorisation and what each can express. Section 3.2 covers the standards that apply to health information systems, which between them address identification, authentication, transport and audit. Section 3.3 covers threat modelling. Section 3.4 covers how access control is verified, which is where the main result of this project lies.

3.1 Access Control Models: RBAC and ABAC

Access control decides which users may do what, and to which records. Two models are treated as the classic approaches, and FHIR's security guidance describes both for health systems [3]. In role-based access control (RBAC), permissions are grouped into roles, each role describes a job function, and users are given roles. If a user's role holds the permission, access is granted [3]. Permissions follow the job rather than the person, which suits a hospital,

where roles such as surgeon, anaesthetist and finance officer already exist before any software is written.

In attribute-based access control (ABAC), the decision is made from attributes and conditions rather than from roles alone. The National Institute of Standards and Technology (NIST) describes ABAC as a method in which authorisation is determined by evaluating attributes of the subject, attributes of the object, the requested operation, and in some cases the conditions of the environment, against rules that describe which operations a given set of attributes allows [4]. NIST places ABAC on a spectrum of logical access control that also includes access control lists and role-based access control, so it is a development of earlier approaches rather than a replacement for them [4]. The practical gain is that rules may refer to the record itself. “A surgeon may create a case” is a role rule. “A surgeon may edit this case” is not, because the answer depends on the case, who it is assigned to, not on the surgeon.

Both standards recognise that a decision may also depend on the circumstances of the request, and both make room for it in their models. NIST calls this environment conditions and lists it as one of the four inputs to a decision, alongside rules, subject attributes and object attributes [4]. FHIR uses the term transaction context and names, among its examples, the workflow state of the request [3]. The workflow position of a case is therefore recognised in both, though under different names and in different parts of the model.

Work on access control in health systems has followed the same direction. A systematic review of twenty studies on electronic health record systems found attribute-based access control to be the most common authorisation mechanism, and grouped the mechanisms proposed into identification, authentication, authorisation and accountability [2]. The review also names what the literature has not resolved: multi-factor authentication, emergency access, patient consent and accountability are identified as gaps, with accountability addressed by only two of the twenty studies. Others have combined the two models directly. Atlam and Yang propose a healthcare model integrating role-based categorisation with attribute-based control over user attributes and environmental factors, observing that models which allocate permissions by predetermined roles or policies adapt poorly to the changing circumstances of a working healthcare system [5].

The three inputs are not alternatives. Roles carry the decisions that follow the job, attributes carry those that depend on the individual record, and the case's position in the workflow carries those that depend on how far it has progressed. What neither standard describes is how they combine in practice, what it costs to enforce them consistently, or how such rules can be checked once built. Chapters 5 and 6 report that.

3.2 Security Standards for Health Information Systems

Health information systems carry data that is sensitive in ways ordinary business systems are not. FHIR's security guidance was the health-specific source used here. It is unusual in addressing implementation directly: it recommends OAuth for authenticating users, states that production data should be sent over TLS, describes how audit events should be recorded, and sets out how a server should respond when access is refused [3].

No Cambodian standard exists for health information security that a hospital can adopt. The nearest domestic guidance is the National Bank of Cambodia's Technology and Cyber Risk Management Guidelines, issued in January 2026 [6]. These apply to banks rather than hospitals, so this system is not subject to them. They are, however, the only Cambodian document setting out detailed security requirements for systems holding sensitive data, and the requirements used here concern the protection of that data rather than financial services.

The guidelines require role-based access control with least privilege, a maintained access control matrix, segregation of duties so that no single person can enter, authorise and complete a transaction, audit logs that record both successful and refused access and cannot be altered, encryption of data in transit and at rest, and vulnerability assessment using both automated tools and manual techniques [6]. They also note that Cambodia's adoption of technology is outpacing its regulatory frameworks, and direct institutions to adopt recognised standards where no local regulation exists [6].

Two general frameworks are relevant alongside FHIR: the NIST Cybersecurity Framework, which organises security work into six functions (Govern, Identify, Protect, Detect, Respond and Recover) [7], and the OWASP Top 10, a list of the ten most common categories of web application security risk [8]. Chapter 4 describes how each was applied.

OWASP also publishes the Application Security Verification Standard (ASVS), a checklist of individual, checkable requirements rather than broad categories. Version 4.0.3, released in 2021, sets out roughly 280 requirements across fourteen chapters, covering areas the Top 10 does not break out on its own, such as session management, error handling, and business logic. It also defines three verification levels, from a basic minimum to the most rigorous, so a system can be checked against a level appropriate to what it handles. A newer version, ASVS 5.0.0, was released in May 2025 and is described by OWASP as its most substantial revision to date [9]. OWASP updated the Top 10 itself in 2025, reordering its categories [10].

Two standards specific to healthcare were also considered. The Health Insurance Portability and Accountability Act (HIPAA) sets security requirements for healthcare data in the United States [11]. ISO/IEC 27001 sets requirements for an organisation-wide information security management system. Its companion, ISO 27799, adapts those requirements to health information specifically. It cannot be used on its own; it still requires ISO 27001 as its foundation [12].

None of these standards states who in a particular hospital may perform a particular action on a particular case. Each addresses authentication, data protection, or general code quality, but none reaches down to a specific organisation's division of responsibility. Those rules have to be drawn from the workflow itself, which is where Chapter 5 begins.

3.3 Threat Modelling Approaches

Threat modelling is the practice of working out what could go wrong with a system, without testing it. It asks what someone might attempt against each part of a system, and what would follow if they succeeded. OWASP treats it as a design activity. Its category for insecure design calls for greater use of threat modelling in authentication, access control and business logic, and separates design flaws from implementation defects, on the grounds that a design flaw cannot be fixed by careful coding [8].

STRIDE is the most widely used method. It groups threats into six kinds [13]: spoofing, where someone acts as another user; tampering, where data is changed without permission; repudiation, where a user denies an action and no record contradicts them;

information disclosure, where data reaches someone who should not see it; denial of service, where the system is made unavailable; and elevation of privilege, where a user gains rights they were not granted. Each part of a system is examined against all six.

The value of the method is coverage, not discovery. Working through six categories for every component makes it harder to miss a whole class of threat. What STRIDE does not do is say which threats matter most, or supply the rules that would prevent them. It can tell you a system may be vulnerable to tampering. It cannot tell you which staff member should lose the ability to edit a record once another department has reviewed it. That judgement comes from the workflow, not from the method.

Threat modelling is usually described as a design-time activity, carried out before a system is built. It can also be applied to a system that already exists. In that case it organises and classifies what is known, rather than predicting what has not yet been found. Chapter 4 describes which of these applies to this project.

3.4 Automated and Manual Security Verification

Writing down who may do what is one task. Checking that the software actually does it is another. A rule table states what should happen, but it says nothing about what the code does. The two can disagree without anyone noticing, because when a system allows something it should have blocked, nothing appears to go wrong. Not every strand of this work presents the same difficulty. Dependency versions, container images and secrets left in a repository can each be checked by tools, because each has a recognisable form. Access control does not, which is why it is the strand this section concentrates on.

Automated tools are the usual first step, and their main difficulty is well documented. Sun, Xu and Su point out that access control rules are specific to each application [14]. Unlike an injection flaw, which looks much the same in any program, there is no general pattern for a tool to search for. The rules have to be supplied, and writing them by hand is slow and tends to produce rules that are missing, incomplete or wrong. Their own tool works around this by inferring the rules from the source code: it builds a list of the pages each role can open, compares those lists to find pages only some roles should reach, and then tests whether other users can reach them anyway.

Inferring the rules from code is where the method reaches its limit. Zhu and colleagues tested six open source applications and found that the assumptions built into earlier tools held those tools back [15]. They tried a different approach, asking developers to supply the rules and turning those answers into patterns the tool could then apply. This found twenty vulnerabilities the automatic methods had missed. A survey published in 2024 reaches the same conclusion, comparing the methods available and listing the problems that remain unsolved [16]. The pattern across all three is that access control rules come from the organisation rather than from the code, and that a tool performs better once a person supplies them.

This is why automated analysis cannot be the whole of the work, and the guidance says as much. OWASP asks developers to write tests for access control [8]. The National Bank of Cambodia’s guidelines require both automated tools and manual techniques [6]. Neither explains how to test permissions that change as a case moves from one stage to the next. Chapter 6 reports what each method found when both were used on the same system in the same period.

Chapter 4

Methodology

This chapter describes how the security work was carried out. It covers how the work was organised, which frameworks and standards were applied and how far, the threat model, and the tools used.

4.1 Research Design

I planned the security work in seven steps: planning and requirements, choosing tools, design, building, testing before deployment, deployment, and checking afterwards. These were grouped under the functions of the NIST Cybersecurity Framework 2.0, and Figure 4.1 shows them. Five were completed at the time this was written; deployment, carried out by a teammate, had not fully succeeded, and the checks that follow deployment had not begun. Deployment has since been completed, every fix verified locally during this work is now live on the hospital's server, though the post-deployment checks (Section 7.3) remain outstanding. The five phases that ran did not run once each in order. Every time I checked something, I found a reason to change a decision I had already made, so the design, the build and the testing each came round more than once. That is why I call the method build-and-check cycles. The seven steps say what I planned; the cycles say how it actually ran.

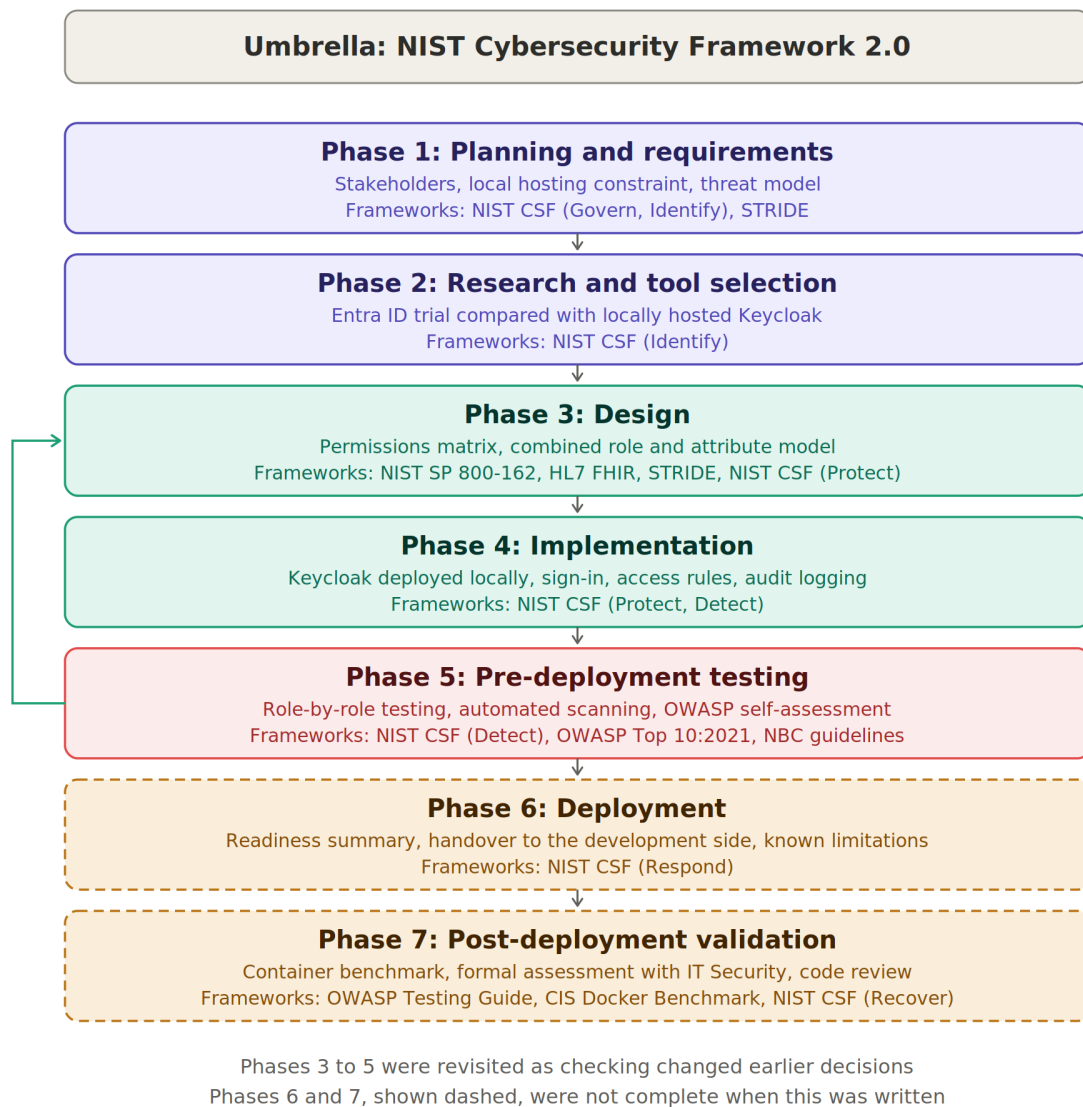


Figure 4.1: The seven phases of the security work, mapped to the functions of the NIST Cybersecurity Framework 2.0. Phases 3 to 5 were revisited as checking changed earlier decisions. Phases 6 and 7 were not complete at the time of writing.

The work covered four things: signing in, what each user may do, how their actions are recorded, and the security of the software the system runs on. Going round more than once mattered. I could not check the rules until I had written them, and I could not write them correctly until I had tested the system.

I did not begin from a written table. The code already existed, written by my teammate, so I started by reading it and writing down what it allowed each role to do. That gave the first version of the rules document. I then went through it against the workflow the

team needed, adding rules and changing others, and writing down every difference I found. Some of those differences I could settle myself. Others needed the team to decide. A few are still open, because they depend on how staff want to work. Differences where the code was simply wrong were listed separately from differences where nobody had decided yet. Both lists are in the rules document. This review was the first of three checks.

The second check was manual testing, role by role, on the running system on my own machine. I signed in with one account for each of the five roles and followed a case from creation through to the insurer's decision. Where the system behaved differently from the rules, I recorded a finding. I then either fixed it, left it with a written reason, or closed it because the system was right and the rule was wrong.

The third check used automated tools on the same version of the code, in the same period: one for the packages the system depends on, one for the container images, one for passwords left in files, and one for the team's own code. I ran the scanners and the manual testing on the same version at the same time, on purpose. It means the two sets of findings can be compared directly, rather than across a version that had changed in between.

A fourth pattern ran alongside these three. I passed changes to the teammate handling deployment, and when a problem turned out to be on the security side, it was reported back to me, I corrected it, and passed it on again. Some problems only appeared once the system was installed, and those reached me through those reports rather than through my own testing.

Each cycle produced documents as well as code changes: the access control rules table, a record of each scan, an assessment against all ten OWASP categories, and a summary of what remained open and who owned it. I wrote these while working, not afterwards, and they are where the numbers in Chapter 6 come from.

I used AI tooling to help write and investigate code, under my direction. What to examine, whether a finding was real, and what to fix or defer were my decisions. I checked what the tooling reported against the running system before accepting it, and several reports turned out to be wrong.

4.2 Frameworks and Standards Applied

I applied frameworks selectively rather than in full. Table 4.1 states which version I used and how far each one was applied, including the two that were planned but not carried out in this phase. Where a newer version has been published since this work was done, the version listed is the one that was current at the time, and re-assessment against the newer version is recorded in Section 7.3.

Table 4.1: Frameworks and standards used, and how far each was applied.

Framework	Version	Extent of application
NIST Cybersecurity Framework	2.0 (2024)	Six functions used to organise the work; control set not implemented
HL7 FHIR	R5	Security section only
NIST SP 800-162	2014	Definition and model of attribute-based access control
OWASP Top 10	2021	All ten categories rated against evidence
NBC Technology and Cyber Risk Management Guidelines	2026	Access control, data security, audit trail and testing sections, used as a benchmark
OWASP ASVS	4.0.3 (2021)	Level 2 target, all 14 chapters assessed, 62 subsections checked
STRIDE	—	All six categories
CIS Docker Benchmark	—	Not applied; planned for a later phase
OWASP Testing Guide	—	Not applied; planned for a later phase

Each source did one job. The NIST Cybersecurity Framework gave the work its structure: the seven phases in Figure 4.1 follow its six functions [7]. The system’s data model already follows FHIR, so FHIR’s security section was the obvious place to look for decisions about signing in and encryption [3]. NIST SP 800-162 gave the access control model [4]. The OWASP Top 10 was the list I checked the code against [8]. The National Bank of Cambodia’s guidelines gave something to measure the result against, rather than a rule to comply with [6]. The OWASP ASVS gave a second, more granular checklist: a separate pass at Level 2 across all fourteen chapters of the 4.0.3 edition, reported alongside the Top 10 results in Chapter 6 [9]. STRIDE gave the six categories used in the threat model described next [13].

HIPAA and ISO/IEC 27001 were also considered. HIPAA was set aside because it is United States legislation, and this hospital is in Cambodia. ISO 27001, and its health-care companion ISO 27799, require building a full organisation-wide information security management system. This includes ongoing risk registers, management review cycles, and certification audits. That scope was too large for a single feature built during a six-month placement. NIST CSF was used instead, selectively, as already described.

Two frameworks in the table were not used. The CIS Docker Benchmark checks how containers are set up and run. The OWASP Testing Guide sets out how to carry out a penetration test. Both belong to work that comes after deployment, neither had been done when this was written, and both are listed in Section 7.3 as work that remains.

I only used parts of each framework, so I cannot say the system complies with any of them. What I can say is which specific requirements were met and which were not, and each one comes from a document I can point to. Read Chapter 6 that way, as findings measured against requirements, not as proof of compliance.

4.3 Threat Modelling

I used STRIDE: spoofing, tampering, repudiation, information disclosure, denial of service, and elevation of privilege, to sort the threats this system faces. I applied it after the system was built, not before. Its job was to file what testing had already found, and to check that no kind of threat had been missed. STRIDE gave the six categories. My findings filled them.

Five of the six hold defects found during this project. The sixth, denial of service, was not addressed, and is recorded as a limitation.

Table 4.2: STRIDE categories and the defects found in each.

Category	Threat in this system	What was found
Spoofing	Someone acts as a user they are not	A user belonging to no group received a valid session and could read every case. Signing out left the identity provider's session open, so the next person at a shared computer entered the previous user's account without a password.
Tampering	Case data changed by someone who should not be able to, or at a point when it should be locked	Doctors could alter the diagnosis and the cost after Finance had reviewed the case. Prices could be changed on cases already submitted or closed. A user could change their own display name, which relabelled every past audit entry for that person.

Continued on next page

Table 4.2 ... continued

Category	Threat in this system	What was found
Repudiation	Someone denies an action, and the record cannot prove otherwise	When the system refused someone, it recorded nothing. Audit entries could not show which role had acted. A refused sign-in is recorded nowhere in the application, because sign-in stops before anything is written. Those attempts appear only in Keycloak's own log, so an investigation has to look in two places. The log stores a reference to the user, and looks up the name when the record is read. So a change of name changes every past entry for that person.
Information disclosure	Data reaches someone who should not see it	A surgeon's list included other surgeons' cases. Finance saw every case by dropping one URL parameter. Every role could read the full staff directory, including email addresses. Removing a patient identifier from the screen did not remove it from the data sent to the browser, where it can still be read. The connection is not encrypted, so patient data and session cookies cross the hospital network in readable form.

Continued on next page

Table 4.2 . . . continued

Category	Threat in this system	What was found
Denial of service	The system is knocked over so nobody can use it	Not addressed in this phase.
Elevation of privilege	A user gains rights they were not granted	Case creation was open to three roles instead of one. Submit and finalise sat with the wrong role. The page-level guard never ran at all. If creating a user's database record fails, the error is logged but the sign-in continues, so a session can exist with no record behind it.

Most of the findings are about people seeing what they should not, or doing what they should not. That is what you would expect from a system where five roles share the same cases. Both kinds are decided by the same three things: the user's role, whether the case is theirs, and how far the case has progressed.

Denial of service was not addressed in this phase. Being reachable only inside the hospital lowers the risk from an outside attacker. It does not remove the risk from a compromised or careless account inside the hospital. Testing for it was not reached in the time available. It is understood to be covered by Canadia Bank (IT security department)'s separate review, though those findings are confidential and not reproduced in this Graduation Project Report. It is recorded in Section 7.3.

4.4 Tools and Technology

The tools fall into two groups: those the system is built from, and those used to check it.

Table 4.3: Tools used in the system and in its verification.

Purpose	Tool	Version
Identity provider	Keycloak	26.7.2
Application framework	Next.js	15.5.24
Session handling	Auth.js (NextAuth)	5.0.0-beta.32
Database	PostgreSQL	16
Database access and migrations	Prisma	7.8.0
FHIR server	HAPI FHIR	7.4.0
Hospital information system	ANZER, read-only	—
Deployment	Docker / Docker Compose	29.7.2 / v5.4.0
Static analysis	Semgrep	—
Container image scanning	Trivy	—
Dependency scanning	npm audit	npm 11.12.1
Secret scanning	secretlint	13.0.2
Browser-driven testing	Puppeteer	25.3.0

Semgrep and Trivy were run in July 2026. I did not fix which version of each tool was used, so both ran whatever release was current that day.

Keycloak was chosen after an extended trial of Microsoft Entra ID. The hospital needed the system to run on hospital-owned servers, and Entra ID runs only as a cloud service. Keycloak also supports user federation with the hospital’s own Active Directory, so staff sign in with the same credentials they already use elsewhere in the hospital, rather than remembering a separate username and password for this system. The application itself never stores or handles user passwords, since sign-in is delegated to Keycloak and, through it, to the hospital’s AD. This also matched FHIR’s security guidance, which recommends OAuth [3], so the choice fit both the hospital’s requirements and the standard the data model follows. Auth.js keeps a user logged in, and the version used is a beta; the makers have not marked it finished, though many production systems use it.

CodeQL and Semgrep both read code looking for mistakes; CodeQL is free only for public code and this project is private, so I used Semgrep, which runs on my own machine.

Each kind of data lives in a different place. Roles are held in Keycloak; the application reads them from the token. Patient records come from ANZER, which this project reads but does not own, change, or assess. Case data and the audit trail sit in the application’s own

PostgreSQL database. The FHIR server is bound to the local interface, so no other machine can reach it.

Chapter 5

Implementation

This chapter describes what was built: signing in and how sessions are handled, what each user may do and how that is enforced, how their actions are recorded, and the security of the software the system runs on.

5.1 Identity and Access Management

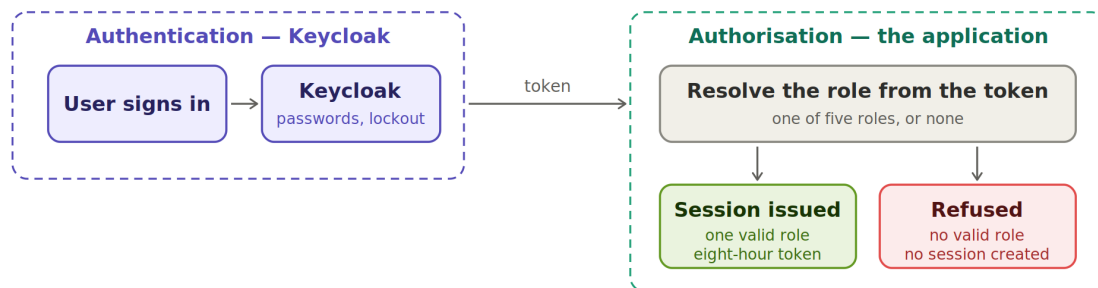
5.1.1 Sign-in and Session Handling

Users sign in through Keycloak, an identity provider running on the hospital's own servers, using OpenID Connect, a standard way for two systems to agree who someone is. The application never sees a password. It sends the user to Keycloak, and Keycloak sends back a token saying who they are and what role they hold. Password rules, account lockout, and the accounts themselves all live in Keycloak.

Roles are held in Keycloak and nowhere else. The application reads the role from the token and keeps no copy of its own. That is why there is no screen for editing permissions: to change what someone may do, an administrator changes their role in Keycloak. Five roles exist: Surgeon, Anaesthetist, Surgical Coordinator, Finance, and System Administrator; every token must match exactly one of them.

If a token carries no valid role, sign-in is refused. The user is not let in with fewer rights, they are not let in at all. This was not how it worked at first: such a user used to get a working session and could reach every page that asked only for a login. The check now

runs in three places: when the user signs in, before any screen appears, and on every request the server receives. The refusal itself is not written to the application's log, because sign-in stops before anything is recorded, so repeated attempts show up only in Keycloak's own log.



The application holds no passwords. Roles are held in Keycloak and nowhere else.
A refusal is recorded only in Keycloak's own event log, because sign-in stops before the application writes one.

Figure 5.1: The sign-in path. Keycloak checks the password and returns a token carrying the user's identity and role. A token with one valid role gets a session; a token with none is refused.

Sessions are stateless: the server keeps no list of who is signed in. When someone signs in, the application gives them a token that works for eight hours. The library's default was thirty days, so I shortened it. Because there is no list, a token cannot be cancelled early; it works until it expires. Keycloak keeps its own session alongside this, ending it after twenty minutes of no activity or eight hours in total. The application adds its own twenty-minute idle timeout, with a warning a minute before. But that runs in the browser, not on the server: a script watches the clock and signs the user out, while the server still accepts a request arriving later. It is there for shared computers, not as a limit on the token.

Two of these gaps were corrected during this work. Signing out now ends both sessions; it once cleared only the application's cookie, leaving the next person at a shared computer in the previous user's account. The Keycloak session lifetimes were also aligned with the application's, so that neither outlasts the other. One consequence of the stateless design remains unresolved: with no server-side session record, a token cannot be withdrawn outright, and idle time cannot be enforced on the server. The specific case of a disabled account keeping a working token is no longer part of this gap, since the live dual-check described below now re-checks status on sign-in and on navigation rather than relying on the token alone, but a

session could still be misused for the remainder of its eight-hour life in ways that check does not cover, since revoking the token itself is still not possible.

5.1.2 Account Status and AD Federation

Keycloak originally held its own standalone accounts, created and managed only inside Keycloak itself, separate from any hospital system. During this work, Keycloak was connected to the hospital's own Active Directory through user federation instead (Section 4.4 describes this decision). Accounts and their enabled or disabled status now come from AD, so a person's access in this system follows the same account AD already manages for them, rather than a separate copy Keycloak held on its own. This made it possible to check status live, on every request, instead of only at the occasional point a standalone account happened to be read.

Access now requires two conditions to hold at once: the account must be enabled in Keycloak, and it must be enabled in the application's own record. The check is centralised in a single function that every entry point calls, so the four surfaces that enforce it (sign-in, page navigation, and the authorisation helpers every API route uses) can never disagree with one another. The two conditions are not symmetric. A Keycloak-side disable overrides everything: if Keycloak reports the account disabled, the user is locked out regardless of what the application's own record says, and no administrator can override that from inside the application. An administrator can still enable or disable an account independently through the application's own screen, and that setting is enforced on its own terms whenever Keycloak's side says enabled: an application-side disable still blocks the user even though Keycloak allows them. So the rule is closer to a logical AND than an override: both sides must say enabled, but only one side, Keycloak, can force a lock-out the other side cannot undo. A further condition covers users who disappear from Keycloak or AD entirely rather than being explicitly disabled: such an account is marked disabled in the application with a distinct "not found" status, and if that person's account later reappears, they are issued a new account rather than having the old one reactivated, a decision made jointly with the hospital's IT team since silently relinking an old account to a returning identity was judged a greater risk than the inconvenience of starting fresh.

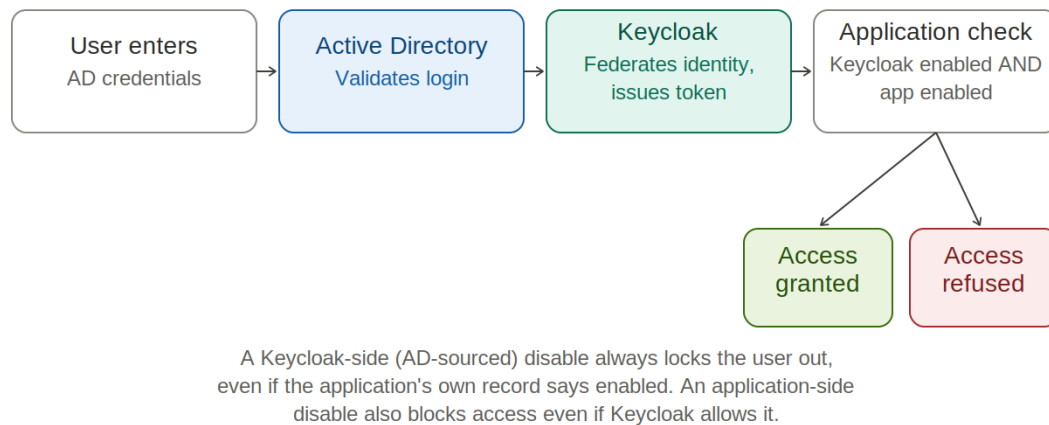


Figure 5.2: Account status after AD federation. Access requires two conditions to hold at once, checked on every request: the account must be enabled in Keycloak, federated from AD, and enabled in the application's own record.

This design replaced an earlier approach that did not work. Account status was originally meant to be enforced through a scheduled job: read each user's status from Keycloak on an interval, and disable anyone Keycloak reported as disabled. That job never ran, because its trigger was written for a hosting platform this deployment does not use, and this was recorded as an open gap.

The live check closes that gap, and does so more completely than the original job would have: a periodic job still leaves a window between checks where a disabled account remains usable, while a live check on every request has no such window. The check does not end an already-open session file the instant an account is disabled: the browser tab stays open, and a request already in flight at the exact moment the account is disabled still completes, which is an unavoidable limitation of any real-time, database-backed check rather than a gap in the design. But the user's very next request to the server, whether that is loading a new page or saving a form section they were in the middle of, is rejected before any write logic runs, with a "account disabled" response, and the refusal itself is written to the audit trail. Actions genuinely completed before that point remain correctly attributed: disabling an account part-way through a session does not alter what was already recorded. The other two background jobs described in Section 5.4, unrelated to account status, were not part of this change and their scheduling remains open (Section 7.3).

The two places were not originally recorded equally. Changes made on the application's screen were written to the audit trail; changes made directly in Keycloak or AD were not, and an investigation into an account's status had to look in two places. This was since closed by configuring Keycloak's event system to forward selected event types into the application's own log: account enabled/disabled, and failed sign-in/sign-out attempts at the Keycloak level. Keycloak's event listener supports far more event types than these; the remaining ones were left as an option for the hospital's IT team to enable later if they judge them useful, rather than added speculatively now.

One finding here mattered more than it first looked. The session carries the identifier Keycloak issues, while the application's own database uses its own row identifiers for the same people. Checks of whether a case belonged to a particular doctor compared the two directly, and two different kinds of identifiers never match, so doctors could not open their own cases. It appeared as several unrelated faults before the single cause was found: a list showing other people's drafts, a saved draft opening read-only, and errors on writing. This was corrected, and the checks now resolve one identifier to the other before comparing.

5.2 Access Control Model

Access control in this system combines the two models described in Chapter 3. Roles carry the decisions that follow a person's job, attributes carry the decisions that depend on a particular case, and the case's position in the workflow carries the decisions that depend on how far the work has progressed.

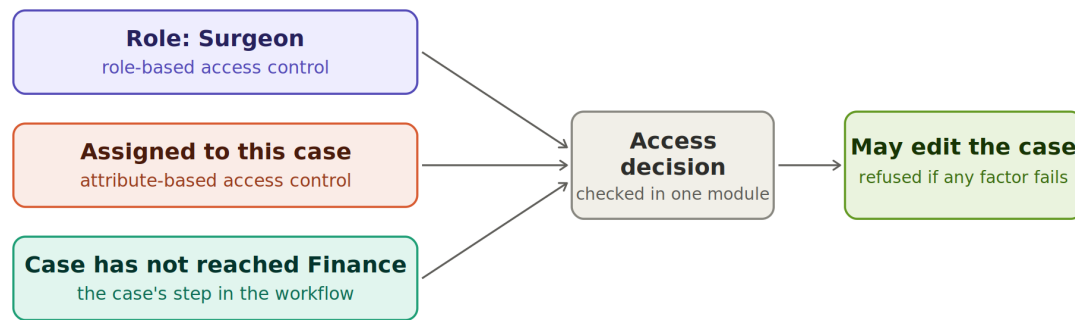
Mapped onto the four inputs NIST describes [4], the model uses three of them. Subject attributes are the user's role, taken from the Keycloak token, and whether their account is active. Object attributes are the identifiers of the doctors assigned to a case and the step that case has reached. The rules are the forty-two action rules described below. Environment conditions are not used: no decision in this system depends on the time of day, the location of the request, or the network it arrives from.

Table 5.1: The five roles and what each is responsible for.

Role	Responsible for
Surgeon	Creating a case, entering the clinical description and their own fee
Anaesthetist	Taking an unassigned case from the pool and adding their assessment
Surgical Coordinator	Monitoring progress, and setting case priority
Finance	Building and confirming the cost, submitting the case, recording patient consent, finalising it, reviewing and sending the request to the insurer, and recording the insurer's answer
System Administrator	Managing user accounts, reading the audit trail, viewing all cases and patient data, assigning doctors to cases, and using emergency override powers (releasing a stuck case, removing documents)

Forty-two actions are defined across the system, and each has a rule saying which roles may perform it and under what conditions. They fall into four groups: eleven for the case lifecycle, six for finance and the insurer, six for documents and notes, and nineteen for reports, administration and reference data. The rules are kept in a separate document, updated as the code changes. It reached its third version during this work, each version following a round of checking.

The three factors work together, not separately. All three must hold, and if any one fails the action is refused. Take one question: may a surgeon edit this case? Using the role alone, the answer is yes, but that would let any surgeon edit any case. Adding assignment narrows it: yes, if the case is theirs. Adding the step narrows it again: yes, if the case is theirs and it has not yet reached Finance. Only the third answer is correct. The step is the least obvious of the three, because it changes the answer for a user who has done nothing: when the anaesthetist finishes their part, the case moves to Finance, and the surgeon becomes read-only. Figure 5.3 shows the three factors as they are evaluated for one request.



All three must hold. If any one fails, the action is refused.
The third can change the answer even when the role and the assignment have not.

Figure 5.3: The three factors evaluated in every access decision. All three must hold; if any one fails, the action is refused.

Underneath these rules is a split the hospital already used on paper. The doctors say what treatment is needed and what their own fee is. Finance builds the final price from that, adding or removing items as the case requires, and commits the hospital to the total. Neither does the other's job: a doctor cannot set what the hospital charges, and Finance does not decide what treatment is given. The case's step is what keeps them apart in software. Without it, a doctor could change a figure after Finance had already settled the price.

One exception existed for a time, and it is worth stating plainly since it did not survive. Early in the work, the Surgical Coordinator kept a narrow right carried over from paper: the ability to adjust a case's pricing after Finance had verified it, on the understanding that the change had been agreed with Finance by phone or in person first. The system could not check whether that conversation happened, so it allowed the edit and only recorded who made it, a trust-based rule rather than an enforced one. This was closed before the work finished. The Coordinator's editing rights were narrowed to their own scope, case priority and response-time targets, and no role now holds an edit right over another role's part of a case on this trust-only basis. The same separation applies to what leaves the hospital; Finance reviews a case before the request is sent, so nothing reaches an insurer without a person having checked it.

Assignment itself needed a rule of its own for the Anaesthetist role. Early on, opening an unassigned case made it the anaesthetist's immediately, with no typing and no saving

required, and there was no way to hand it back if it had been opened by mistake. A case is now taken only when the anaesthetist saves or verifies it, and they can release it while it is still a draft. This matters for the same reason the case's step matters elsewhere: an action with a real consequence, becoming responsible for a case, was previously happening as a side effect of merely looking at something.

Access is decided in one place. A single module checks that the user has a session, that their account is active, and that their role is valid. Where the action needs it, the same module also checks that the case is theirs and has reached the right step. Nearly every address in the system calls it. Not every part uses the full check, though: at least eight places ask only whether the role is right, then work out for themselves whether the case belongs to the user, using their own separate copy of that rule rather than the shared one. This has already caused a real problem once, not just a theoretical risk: one of those copies fell out of step with the shared rule and briefly let a surgeon reach a case they should not have been able to see, before it was found and corrected. The other copies currently still agree with the shared rule, kept in sync only by developers remembering to update every copy by hand. Chapter 6 returns to this.

5.3 Audit Logging

The audit log records every action that changes a case. An entry holds who acted, the role they held at the time, which case was affected, what was done, when, and which part of the system was used. Refused attempts are recorded too, with the reason. The entry does not record the device or the network address, so it can show that an account acted but not which machine was used.

The record is append-only: entries can be added, but never changed or deleted. This is enforced at the database level, through a trigger that rejects any update, delete, or truncate against the table outright, combined with a permission grant that removes the application's own database login's ability to attempt those operations at all. Neither the System Administrator role inside the application, nor a compromised application process using its normal database credentials, can alter a log entry; both are blocked before the operation would even reach the trigger. OWASP recommends exactly this for records that

need to be trusted [8]. It is what makes the log usable as evidence, rather than just something to look things up in.

One honest caveat sits underneath this, distinct from anything reachable through the application: whoever holds the database’s own superuser credentials (a different, infrastructure-level actor entirely from any role the application defines) can disable the trigger with an explicit administrative command before modifying rows, then re-enable it. This is not a gap in the design so much as an inherent property of database superuser access generally, and it requires credentials no application role has access to. It happened exactly once, in a local development database, as an authorised, one-off cleanup of test data, carried out as a deliberate and distinct action rather than an ordinary delete, and never applied to the deployed system. Those credentials were handed over to the hospital at the end of the placement; the development team retains no database access on the live system.

The role is recorded at the moment of the action rather than looked up later. This was corrected during the work: Finance and the Coordinator had both been logged as STAFF, so the record could not show which of them had acted. Stamping the exact role at the time of writing also means that a later change of role does not alter what past entries say.

The administration screen presents a second log alongside it, and the two are not the same thing. The audit log records workflow events, as described above. The activity log records every successful request that passes the access checks, including the request body, so it accumulates patient identifiers and clinical detail as a side effect of recording traffic. It originally had none of the protection the audit log has: its rows could be changed or deleted, and no retention period was set, and this was recorded as an open gap. It has since been given the same append-only design as the audit log, and is in fact slightly stricter: the audit log’s trigger allows one narrow, deliberate exception, where deleting a case correctly nulls out that entry’s case reference rather than leaving a dangling link; the activity log has no foreign-key relationships of this kind, so its trigger rejects every update, delete, and truncate outright, with no exception at all.

Two limits remain. Neither is a fault in the sense of a broken control; the second is a genuine, unresolved inconsistency worth stating precisely. The first is names, and it is more mixed than a single description covers. The audit entry’s actor field is a reference to the user, resolved to a display name only when the record is read, so a later change to that

person's name changes how every past entry involving them is shown. Separately, many entries also embed a name as literal text inside their own detail, "Case finalised by John Tan (SURGEON)," for instance, captured at the moment the action happened, and this text is genuinely frozen: it is written once into the append-only log and the database itself rejects any later change to it. The result is that a single entry can show two different names for the same person if they are renamed in between: the frozen name in the detailed text, and the current name in the actor field next to it, disagreeing with each other on the same row. Users can no longer rename themselves through the application, though names synced in from Active Directory can still change this way, so the inconsistency remains reachable. The same current-name behaviour reaches insurer emails, printed forms and the coordinator export, so a rename can also affect documents that have already left the hospital, independent of the log itself. The second limit is who may read the log. Administrators open it through the administration screen built as part of this work, and their own actions appear in both logs, so they can see what has been recorded about them. Separating those two responsibilities would mean redesigning who can read what, not fixing a fault.

Four things are not written down. A refusal that means only "not yet" (submitting a case before it is ready, or deleting one that has moved past draft) comes from the workflow rather than from a permission check, and only permission refusals are written down. A request for a case that does not exist is refused silently. A refused sign-in is recorded nowhere in the application, because sign-in stops before anything is written, so those attempts exist only in Keycloak's log. And nothing at all can be written when the database itself is down.

Nothing checks the log as it fills, and nobody reviews it on a schedule. Refused attempts are written down correctly, but no one is told, so twenty attempts in a row would sit in the table until somebody happened to open it. OWASP renamed this category in 2025 to include alerting for that reason [10]. Both are recorded in Section 7.3: the software that would raise an alert, and the hospital routine that would review the records.

5.4 Deployment

My teammate deployed the application, and the hospital’s IT department provided the server. What follows is about the security of that environment: what I found in its configuration, what was corrected, and what is still open.

The system runs on a Windows server the hospital owns, reachable only from inside the hospital. Four programs run on it side by side: the application, the database, Keycloak, and the FHIR server. None can be reached from the internet, and the FHIR server can only be reached from the machine itself. Patient data is read from the hospital’s own system, with limited read-only access granted by the IT department, so this deployment holds no patient records of its own. I did not test whether the system keeps running under normal use, and that is recorded in Section 7.3.

The application has a shortcut for developers: skip sign-in, pick a role, no password. It is useful on a laptop and dangerous on a server. The template that people copy when setting up a new server had that shortcut switched on, so anyone following it would have built a server that anyone could walk into. This was fixed twice. The template now has it switched off, and the application refuses to start if the shortcut is on in production.

The browser-facing connection now runs over HTTPS. Two internal connections remain unencrypted: Keycloak to the application, and Keycloak to Active Directory, so credentials and status information passing over those two links still cross the network in the clear. Keycloak also cannot leave its development mode until its own connections are encrypted, and nobody has checked whether the database on disk is encrypted. Turning on the remaining encryption is the server administrator’s job, not mine.

The four containers are third-party software, so rather than assume they were safe I scanned them, and Chapter 6 reports what that found. One problem was found without a scanner. Keycloak had not been told where to keep its accounts, so it had defaulted to H2, the lightweight, file-based database built into Keycloak itself, meant only for local testing and explicitly not recommended by its makers for a real deployment. H2 stores its data as a single file inside the Keycloak container rather than as a separate, durable service, so losing that container would have meant losing every account outright, and the file could not be backed up without shutting Keycloak down first to release it. Keycloak was reconfigured

to use the PostgreSQL database already running alongside it instead, in a separate schema with its own restricted login, so accounts now live in the same durable, backed-up database as the rest of the system rather than a throwaway file tied to one container's lifetime.

Three background jobs existed at the time this was written: the account-disable job described in Section 5.1, one to warn when a case is taking too long, and one to close cases automatically once past a deadline. None of them ran, for the same reason in every case: all three declared their schedule in a file meant for a hosting service this project does not use, and nothing on the hospital's server started them. One was also called from the browser when the application was open, but that call used a method the route did not accept and was rejected before any of it executed.

Each of the three has since taken a different path. The account-disable job was superseded rather than fixed, by the live dual-check design described in Section 5.1. The deadline-closure job was retired outright as a feature rather than scheduled, it closed cases after a flat 30-day window unrelated to the SLA tiers described below, and it was deliberately removed from the codebase; a database migration later confirmed it had auto-closed zero cases across the system's history before being deleted. The slow-case warning job was kept and given a working trigger, but not the one originally planned: rather than an OS-level scheduler such as Windows Task Scheduler, it now runs on an in-process timer inside the application itself, firing automatically every 15 minutes for as long as the application process is running. This closes the "never triggers" problem for as long as the process stays up, though it is a different, less independent guarantee than an OS-level schedule would give: if the application process stops, the timer stops with it, whereas a Task Scheduler entry would not depend on the application process being alive at all.

The rule the slow-case warning job checks against was also revised after this was first written. Cases originally fell into two service-level categories, adjustable from a settings screen. This was changed to three fixed categories: Emergency (1 hour), Urgent (8 hours), and Routine (48 hours), read from a database-backed configuration rather than the earlier hardcoded pair. The permission to change these three thresholds was built for the Surgical Coordinator and System Administrator roles, with validation, a database write, and its own audit entry, and this was tested working.

It is not usable today: the screen that edits it and the route behind it are both switched off behind a single flag, deliberately, after two real bugs were found in the editor during testing, a saved total that was written but never read anywhere, and a dashboard countdown computed from a separate table the editor did not update. Rather than ship a partially working editor, both the screen and the route were locked pending a decision, and the route refuses the request outright regardless of who is asking, so a disabled feature can never be mistaken for a wrong-role refusal in the logs. Reading the current thresholds remains open to everyone; only changing them is switched off. So the thresholds are fixed for every role today in practice, though not by design: the capability to adjust them exists, built and tested, waiting on that flag.

The job raises its warning in stages against these thresholds rather than only at the deadline, and the warning itself is system-level only: it surfaces as a status or flag inside the application rather than being sent out as an email. These tiers apply only to this warning job; the separate deadline-closure feature that once existed worked on an unrelated flat time window and, as noted above, was removed rather than connected to this newer tier system.

Chapter 6

Result and Discussion

This chapter reports what the security work produced, and what those results mean. Section 6.1 gives the findings from both checking methods, the numbers each produced, and the decisions taken on them. Section 6.2 looks more closely at two of those results: how permissions in this system depend on more than the user's role, and why the automated tools did not find the problems that manual testing found.

6.1 Result

6.1.1 How the System Was Checked

The system was checked in two ways. Automated tools looked at three things: the outside packages the system uses, the container images it runs in, and the project's own code. Manual testing was done role by role on the running system, using one account for each role and following each case from creation to the insurer's decision. Both were done in the same period, on the same version of the system, and the rules being tested covered forty-two actions across five roles. The ASVS assessment described in Section 6.1.6 was carried out separately, later in the work, against a subsequent version of the system.

Two checks were still to come at the time of this assessment. The hospital's IT Security Division had not yet tried to break into the system, and the container settings had not yet been checked against the CIS Docker Benchmark. Neither had been done when this was written. Both have since occurred: Canada Bank (IT security department) carried out

a penetration test jointly with the hospital’s IT Security Division, and separately reviewed the project’s code and its containers. Their findings are confidential and are not reproduced in this Graduation Project Report.

6.1.2 Package and Container Scanning

Table 6.1: Scanning targets and findings.

Target	Tool	Findings
npm packages	npm audit	15: 2 critical, 4 high, 8 moderate, 1 low
Keycloak 26.6.4	Trivy	16 high, 0 critical
PostgreSQL 16	Trivy	72, then 55 after an update
HAPI FHIR 7.4.0	Trivy	132: 27 critical, 105 high

Every problem was in software the project uses rather than in code the team wrote: third-party libraries and container images. Updating the login library cleared both critical findings, taking the count from 15 to 13 with none critical. Eight of the remaining problems were in the application framework, and looked at first as though they needed a major upgrade. They did not: moving from version 15.5.20 to 15.5.22 was enough. One package, sharp, is still below the version its advisory asks for, because the fix offered would have undone the framework update. That was judged a step backwards, and it is recorded as an open item.

Updating the database image reduced its findings from 72 to 55, and its critical findings from 19 to 17; the rest have no fix available yet, so nothing can be installed. The FHIR server image was the worst of the three, and whether it is needed at all was raised as an open question for the team rather than treated as something to patch. Keycloak was also found to be keeping every account in its own built-in database, which its makers say not to use for real systems. It was reconfigured to use the PostgreSQL database already running alongside it, in a separate area with its own restricted login. The accounts on my own machine moved across keeping their original identifiers; any other installation starts with an empty Keycloak and needs its accounts created again.

6.1.3 Checking the Project's Own Code

Semgrep was used for this step; Section 4.4 explains the choice over CodeQL. It applied 257 rules across 356 files and returned 15 findings. secretlint was also run over the repository and its files, checking for secrets left in the code such as passwords or API keys; it found none.

Table 6.2: What Semgrep found.

Type	Count
Path traversal, looked at separately	1
Regular expression built at run time	3
Unsafe text in log messages	5
Unpinned action versions	4
Installer piped into a shell	1
Package setting	1

None of the common problems appeared. There was no SQL injection, no cross-site scripting, no password left in the code, and no way around the login. Nothing was about access control.

The path traversal was looked at more closely and could not be used. The database check runs before any file is written, and an invalid case identifier fails there, so the write is never reached. That protection was accidental rather than planned, so it was recorded as something to make deliberate: check the identifier properly, and mark the order of steps as security-critical so a later change does not remove it by accident.

6.1.4 Manual Role-by-Role Testing

What Was Found

A finding was counted as access control if it changed which users could do which action in which case. Findings about audit logging, sessions, input checking, data quality, and screen behaviour were counted separately. The logout defect was counted as access control rather than as a session problem, because its effect was that one user reached another user's account without logging in. On this basis the security phase recorded 47 findings, of which 23 were about access control.

Table 6.3: The 47 findings by type.

Type	Count	Example
Access control	23	See Table 6.4 for the six sub-categories
Screens, data quality, workflow	16	Field validation, labelling, and workflow-order issues not related to permissions
Audit and logging	4	e.g. refused actions not written to the log
Input checking	1	A single validation-related finding; specific detail not retained
Sessions	1	Logging out left the Keycloak session open, letting the next user at a shared computer in without a password (fixed)
Deployment and configuration	1	Keycloak had defaulted to its own built-in H2 database instead of the PostgreSQL database alongside it (fixed)
Closed, not a defect	1	A finding investigated and confirmed as intended behaviour, not a bug; which one is not retained

The 23 access-control findings fell into six groups.

Table 6.4: The six kinds of access-control finding.

Group	Example
Seeing cases they should not	A surgeon's list showed other surgeons' drafts
Actions held by the wrong role	Submit and finalise sat with the Coordinator, and refused Finance, the role that owns them
No lock on the case's step	Doctors could still edit details and costs after Finance had reviewed the case
Wrong identifier compared	Assignment checks compared two different kinds of user id, so doctors could not reach their own cases
Letting people through on failure	The disabled-account check allowed access when it found no record
Sessions outliving logout	Logging out left the Keycloak session open, so the next person at a shared computer entered the previous user's account

What Was Done About It

Most findings were fixed before deployment. Twenty-three improvements were recorded as decisions to leave for later, each with a stated reason and an owner.

Table 6.5: Examples of work left for later, and why.

Left for later	Reason
Duplicate drafts	Detection is built and working: a surgeon starting a case is warned of an existing draft and can jump to it. Deleting the old draft is possible but through a separate, general delete action, not linked to that warning; merging two drafts into one was never built
Reversal (formerly “appeal”)	Built as a complete feature, allowing Finance or an administrator to reopen a closed case and re-enter the workflow without starting over, but switched off behind a single flag pending a decision to enable it
Refused sign-ins not recorded in the application log	Sign-in stops before the audit write; the attempts exist in Keycloak’s log

The clearest illustration of a finding that carried more weight than it first appeared to have concerned the page-level access rules. The file holding those rules sat at the top of the repository, while the application runs from a folder inside it. The framework only looks inside that folder, so it never found the file, and those rules had never run at all. Nothing was exposed, because the central module described in Section 5.2 was checking every request anyway. What only emerged later was that the file’s own settings excluded those addresses entirely, so even had it loaded, it would never have guarded what it appeared to guard. It read as a control, and was never capable of being one. It was left as it stood: switching on

a gate that has never run means every page might behave differently, and that needs a full test pass across the whole system.

A related finding was fixed at once. A user who belonged to no group still received a working session, and could reach any page and any address that only asked whether someone was signed in, including a case list that returned every case. Sign-in is now refused when no valid role is found.

Two findings concerned the point at which work should stop, and both were fixed. Doctors could change the diagnosis and the cost after Finance had reviewed the case, without Finance knowing, so Finance had committed to a figure the case no longer matched. Editing now stops the moment a case reaches Finance. This is the finding that shows the case's step has to be part of the rule: the doctor's role has not changed, and the case is still theirs, but the answer must change anyway. Separately, opening an unassigned case made it the anaesthetist's immediately, with no typing and no saving, and there was no way to hand it back; this is described in Section 5.2, where the take/release rule that fixed it is set out alongside the rest of the access control model.

Logging out cleared only the application's own session while the Keycloak session stayed open, so the next person at a shared computer entered the previous user's account with no password. Signing out now ends both.

The log had a gap too: it was written only after a successful action, so a refused attempt left nothing behind. This was partly closed. Refusals where someone reaches for a case that is not theirs are now recorded, with the person, their role, the case and the reason. Refusals that only mean "not yet" are still not recorded, and neither is a deletion refused because the case has moved past draft.

A further gap sits earlier. When someone with a real account but no role is refused at sign-in, nothing is written to the application's log at all, no success, no failure, because sign-in stops before the point where anything is recorded. Those attempts appear only in Keycloak's own log, which is switched on, so an investigation has to look in two places rather than one.

Two further weaknesses concern the audit trail itself. Records hold a reference to the user rather than the name as it stood at the time, and look the name up when the record is read, so a rename relabels every past entry for that person, including sign-ins and

refused attempts. Separately, administrators can open the audit trail while their own actions are recorded in it, so a privileged user can see what was captured about them. Both are design changes rather than fixes, and both are recorded for later. Neither would be found by a scanner, and role-by-role testing did not flag them either, because the system behaves exactly as it was built to.

Two findings concerned patient data, and neither has been resolved. Removing a patient identifier from the screen did not remove it from the data sent to the browser, where the identifier, the national ID and the phone number were still readable. Fixing this properly means sending only what each row needs and moving the search to the server, which was judged too large a change before go-live.

The second concerned transport. At the time, the system ran entirely over plain HTTP, which the browser reported directly: it showed the connection as not secure. The browser-facing connection has since been put behind HTTPS. Two internal connections remain unencrypted: Keycloak to the application, and Keycloak to Active Directory. Patient data and credentials passing over those two links still cross the hospital network in the clear. This is a partial departure from a standard the project otherwise follows, since FHIR's security guidance states that production data should be sent over an encrypted connection [3]. Encrypting the remaining two connections is a server configuration rather than an application change, and it is recorded in Section 7.3.

Two further findings came to light during the final review, after the counts above were settled. The first concerns a second log, already introduced in Section 5.3: the activity log accumulates patient identifiers and clinical detail as a side effect of recording traffic, and at the time it had no protection against being changed or deleted, unlike the audit trail. It has since been given the same append-only protection as the audit trail. The second concerns the background jobs described in Section 5.4: three existed, and at the time none had ever run. Each has since taken a different path. The account-disable job was superseded by the live dual-check design described in Section 5.1. The slow-case warning job was kept and given a working trigger, an in-process timer rather than the originally planned OS-level scheduler. The deadline-closure job was retired outright rather than scheduled.

6.1.5 OWASP Top Ten Assessment

Every category of the OWASP Top 10:2021 was rated against evidence from this project. Categories were rated strong where the risk was assessed and no significant gap remains, partial where controls exist but a known gap is recorded, and needs work where a material weakness is unresolved.

Table 6.6: OWASP Top 10:2021 ratings.

Rating	Categories
Strong	A07 authentication
Partial	A01 access control, A02 cryptography, A03 injection, A04 design, A05 misconfiguration, A06 outdated components, A08 data integrity, A09 logging, A10 request forgery
Needs work	—

No category was rated as needing work, though several of the partial ratings hold real, current gaps. A01 was closed by the role-by-role testing described above rather than by a scanner, and most of it holds up well: object-level scoping and the case-assignment checks were fixed, and the Coordinator’s access was narrowed to its own scope. It is not rated strong, because System Administrator currently has write access to clinical form data and cost figures through two routes, despite the system’s own design stating this role should be view-only there; this is tracked as an open item in Section 7.3.

A02 covers encryption in transit and at rest: the browser-facing connection now runs over HTTPS, but two internal connections, Keycloak to the application and Keycloak to Active Directory, remain unencrypted, and nobody checks whether the disk is encrypted. A05 covers configuration hardening. Real security headers are set, and required secrets fail loudly rather than falling back to an insecure default, but Keycloak runs in its development mode even in the production deployment path, which its own documentation advises against, and no automated dependency or code scanning runs in the build pipeline to catch drift between releases. A fourth A05 item was fixed during the phase: Keycloak’s built-in database, which was replaced by the PostgreSQL database already running alongside it.

A09 rests on an audit trail that cannot be changed or deleted, protected by the database itself, and that records refusals along with the reason. At the time of this assessment, two gaps sat against that: nothing watched the log as it filled, and the activity log

held request bodies without the same protection as the audit trail. The activity log has since been given the same append-only protection, closing that specific gap. What remains is narrower but still real: a sign-in refused for having no valid role writes nothing to either log, only to a server console outside the application's own record, and nothing watches either log as it fills.

A06 rests on scanning and fixing packages and container images, and was strong immediately after that work. It did not stay that way: a later dependency check found new advisories that were not present right after the fix, because no automated scanning step runs in the build pipeline to catch this drift between releases. The components in active use are still current; what is missing is a repeatable check, not a one-time fix, and that gap is recorded in Section 7.3. A07 relies on Keycloak, the password rules, account lockout and session handling, now strengthened further by completed AD federation and a live per-request account-status check; there is no multi-factor authentication, which was deliberately left to a later phase, but nothing found weakens this rating.

6.1.6 ASVS 4.0.3 Assessment

Alongside the OWASP Top 10 review, the system was separately assessed against the OWASP Application Security Verification Standard, version 4.0.3, targeting Level 2 [9]. ASVS is a more granular standard than the Top 10: rather than ten broad risk categories, it sets out roughly 280 individual requirements across fourteen chapters, covering areas the Top 10 does not break out on its own, such as session management, error handling, and business logic.

A newer version, ASVS 5.0.0, was also available at the time, released in May 2025, but it was not used, for two reasons. First, its biggest change was making the standard's easiest level, Level 1, simpler to meet. That did not matter here, since this project was always aiming for the stricter Level 2. Second, its other major additions covered cloud-native systems and WebRTC, technologies this project deliberately does not use, since the system runs entirely on its own hospital-hosted server. One part of ASVS 5.0 did seem relevant, its improved coverage of OAuth and OIDC, the login standards this system relies on, so that is kept as a reason to check the system against ASVS 5.0 in the future, noted in Section 7.3.

All fourteen chapters were assessed; none were skipped wholesale, though a small number of individual requirements were marked not applicable where the underlying feature does not exist in this system, for example, the requirements covering SOAP and GraphQL interfaces, since neither is used here, and self-service credential recovery, since accounts are provisioned rather than self-registered. Sixty-two subsections were checked in total: 29 were met, 20 partially met, 6 not met, and 7 not applicable. Level 2 was not fully achieved; no chapter was rated as fully met.

Table 6.7: ASVS 4.0.3 results by chapter.

Chapter	Met	Partial	Not Met	N/A	Subsections
V1 Architecture, Design and Threat Modelling	5	3	1	0	9
V2 Authentication	2	3	0	2	7
V3 Session Management	5	1	0	1	7
V4 Access Control	3	0	0	0	3
V5 Validation, Sanitization and Encoding	2	2	0	1	5
V6 Cryptographic Storage (at Rest)	1	1	1	1	4
V7 Error Handling and Logging	0	3	1	0	4
V8 Data Protection	1	1	1	0	3
V9 Communications	1	1	0	0	2
V10 Malicious Code	2	1	0	0	3
V11 Business Logic	0	1	0	0	1
V12 Files and Resources	5	0	1	0	6
V13 API and Web Service	0	2	0	2	4
V14 Configuration	2	1	1	0	4
Total	29	20	6	7	62

Each of the six not-met subsections sits in a different chapter, with no chapter failing more than one: no documented threat-modelling process or automated security scanning in the build pipeline (V1); patient data held in plain, unclassified columns with no field-level encryption (V6); no centralised log monitoring or alerting (V7); no data-retention or deletion policy (V8, the same gap noted below as deliberately deferred); no malware scanning or integrity check on uploaded files (V12); and minor server-fingerprinting headers left at their defaults (V14). Taken together, the weakest areas cluster around process and lifecycle gaps rather than exploitable code-level defects: nothing here is a working attack, but each is a control the standard expects that this project does not yet have.

Fifteen findings were tracked (F1 through F15). Ten were resolved or substantially addressed during this work, including a critical Keycloak vulnerability allowing account takeover through a flaw in its credential-reset flow, fixed by upgrading the Keycloak version; eight high-severity dependency vulnerabilities, patched to bring the dependency scan to zero high or critical findings at the time; patient identifiers that were being sent as URL query parameters rather than in the request body, moved across four call sites; and an access-control gap on the patient-lookup endpoint that had no case-assignment check at all, closed by routing it through the same shared visibility check used elsewhere in the system. That dependency result did not stay at zero: a later check found new high-severity advisories in build-time tooling, disclosed after the original fix rather than caused by any reversal of it. No automated scanning step runs in the build pipeline to catch this kind of drift between releases, so a point-in-time fix does not stay verified on its own; this is recorded in Section 7.3.

Three findings were left open at the time of the assessment. One concerns error messages that can leak internal detail to the browser rather than being suppressed outside of development. One concerned a developer sign-in shortcut that bypassed the normal login entirely. One concerns the lack of a step-up re-authentication requirement before high-consequence actions such as finalising or sending a request to the insurer. The remaining two are carried into Section 7.3.

The developer sign-in shortcut was narrowed rather than fully removed. A loophole that let it activate outside a developer's own machine, on a staging server despite a production environment setting, was closed. The underlying shortcut itself is still present in the

codebase, gated behind a flag that defaults to off and only takes effect in a non-production environment, such as a developer running the system locally. It is not reachable in the deployed system as configured, but it is not gone, and a future configuration change that enabled the flag outside a developer’s own machine would bring it back. This is recorded as still open in Section 7.3, not as resolved.

Two further findings were deliberately deferred rather than treated as defects. The first is the dead page-rule file already discussed in Section 6.2.3: the unused file itself was removed, but building the real route-level enforcement layer it was meant to provide was set aside, since enabling nineteen untested rules across five roles risked silently locking a working role out of something it needs, without a full test pass to catch that first. The second is the absence of any data-retention or deletion policy, recorded here as what it is, an organisational decision for the hospital to make, not something a fix to the code could resolve on its own.

The assessment was carried out with AI assistance (Claude Sonnet 5), under my direction, consistent with the approach described in Section 4.1: what to check, whether a finding was genuine, and what to fix or defer were my decisions, verified against the running system rather than accepted as reported.

6.2 Discussion

6.2.1 Permission Depends on More Than the Role

Role-based access control asks one question: what is this user’s role? In this system that was never enough on its own. Every rule rested on three things together, and removing any one of them gave the wrong answer.

Table 6.8: The three factors, with an example of each.

Factor	Example from this system
Role	Only a surgeon may create a case
Assignment	A surgeon sees only their own cases; a doctor may remove only files they uploaded
The case’s step	Once a case reaches Finance, both doctors become read-only

The third factor is the easiest to miss, because it changes the answer for a user whose role and assignment have not changed at all. A surgeon may edit their own case one day and

not the next, not because anything about the surgeon changed, but because the anaesthetist has since finished their part and the case has moved to Finance. Neither the surgeon nor Finance did anything. A third person completing their work changed what the surgeon was allowed to do.

This was found as a real defect: doctors could change the diagnosis and the cost after the case had reached Finance, and Finance did not know.

That matters more than a single wrong permission, because it undoes a rule the hospital depends on. The doctors say what the treatment costs and Finance authorises it, and neither does both. If a doctor can change the figure after Finance has approved it, the separation exists on paper only. The case's step is what keeps the two apart.

The lock has a cost, and it was accepted knowingly. Once both doctors have verified, they are read-only. So if a correction is needed later, if the insurer asks for more clinical detail, for instance, the doctor cannot supply it directly, and the change has to go through Finance or the coordinator. Uploading a document stops at the same point. The team judged this acceptable for a first phase, because the alternative is letting clinical data change underneath a price Finance has already confirmed. Whether to loosen the lock, and how, was left until after the go-live test.

Permission in this system depended on three things together, and the case's step was the one most easily overlooked. Most of the twenty-three access-control findings turned on assignment or on the case's step, not on the role.

6.2.2 Why the Scanners Found No Access-Control Problems

Semgrep applied 257 rules to 356 files and returned 15 findings, none of them about access control. This is not a failure of the tool.

A rule in a scanner describes the shape of code. It can spot a query built by joining text together, or a password written into a file, because those look a particular way wherever they appear. "A coordinator may not submit a case" has no shape. It is a decision the hospital made about who is responsible for what, and nothing in the code marks it out from any other function call. A scanner cannot check a rule it was never given.

What this project adds is a measurement from a working clinical system. On the same code, in the same period, a general-purpose scanner found nothing in this category while role-by-role testing found twenty-three problems, several of them serious. Zhu and colleagues report the closest comparison: automatic analysis of six applications found a set of vulnerabilities, and when developers gave the tool the application's own access control rules, twenty more appeared that the automatic methods had missed [15].

A related gap appeared before any scanning was done. The access control rules were not written before the code; they were read out of it, because the code came first. Setting that description against the workflow the team needed showed that a substantial number of rules did not match what the hospital wanted, and that some expected actions did not exist at all. Sun, Xu and Su note that access control rules are specific to each application, which makes it hard to get a precise specification for a tool to enforce [14]. Here there was no specification to enforce against: it had to be recovered from the code first.

The consequence is short. A clean scan is evidence about code patterns. It is not evidence that the right people can do the right things, and it should never be recorded as though it were.

6.2.3 Layered Checks, and the Guard That Never Ran

The clearest illustration of layered checking came from a mistake. The file holding the page rules sat outside the folder the framework reads, so it never loaded, and that whole layer had been doing nothing for the life of the project. Nothing was exposed, because the central module was checking every request anyway. What only emerged later was that the file's own settings excluded those addresses entirely, so even had it loaded, it would never have guarded what it appeared to guard. It read as a control, and was never capable of being one.

Two things follow. Nobody knew the layer was dead until it was found by accident, which is an argument for testing that a control actually runs rather than assuming it does because the code exists. A protection that has never run offers nothing, and it looks exactly like a working one until someone checks. The system survived because the decision was made in one place, which is what OWASP asks for, that access control be implemented once and

reused throughout an application [8]. That is not complete: at least eight other places in the code write their own separate copy of the same case-assignment rule instead of calling the shared check. All eight currently agree with each other, but only because each copy has a comment telling a developer to keep it matching the others by hand, nothing in the code forces them to match. This already went wrong once: one of those copies, in the part of the system that lets an anaesthetist open a case, fell out of step with the real rule, and for a time let a surgeon reach a case they had no right to see. It was found and corrected, but nothing stopped it from happening, and nothing stops the same drift from happening again in one of the other seven copies. Rules kept in more than one place drift apart, and this one already has; this is recorded in Section 7.3. The same inconsistency shows up in reverse elsewhere in the codebase: the route that manages user accounts checks permission before ever touching the database, so a non-administrator is blocked before the system reveals whether a given user ID is even real, the correct pattern already exists, it simply was not applied everywhere.

The same pattern appeared twice more. In the path traversal finding, an attack failed only because a database check happened to run first; the system was safe, but not for the reason anyone had designed, and changing the order of steps later would have removed that protection without warning. And three background jobs were written, correct, and never ran: nothing on the deployment server started them, and the one call that did reach a job used a method the route did not accept, so it was rejected before anything executed. Each has since taken a different path. One, the job meant to close the gap left by a session that cannot be withdrawn, was later superseded rather than fixed: a live check replaced it rather than the job itself being scheduled, which is a different way of closing a gap than the pattern this section is about, but it means that particular gap is no longer open. Another, the slow-case warning job, was given a working trigger, an in-process timer rather than the originally planned OS-level scheduler, so it now runs as intended, if by a less independent mechanism than first planned. The third, the deadline-closure job, was retired outright rather than fixed or scheduled.

6.2.4 What This Suggests for Similar Systems

Four points carry beyond this project.

Where a workflow hands responsibility from one group to another, the state of the work belongs in the permission rule alongside the role. Without it, the handover is not enforced at all.

Where authorisation rules come from an organisation rather than from the code, they have to be tested by using the system as each role, because no scanner has access to them.

And where a control matters, it is worth confirming that it runs. Code that exists and code that executes are not the same thing.

Where a system's own rules say what a role should not be able to do, that has to be checked directly against the code, not assumed from a clean scan or from what the screens allow. A role's real reach can extend past its visible interface to routes with no button behind them, invisible to both a scanner and to testing done through the screens; catching that gap takes a third check, one that compares the code to the system's own stated rules rather than to what a scanner or a screen would show.

Chapter 7

Challenges, Conclusion and Future Work

This chapter covers the difficulties encountered during the security work, the conclusions drawn from it, and the work that remains for later phases.

7.1 Challenges

7.1.1 Technical Challenges

I started with no background and no map. I had not built software before this project, so reading someone else's code and changing it safely were things I learned while doing them. Nor was there a structure to follow. At the beginning my learning was scattered, one topic, then another, in no order. What structure existed came later, from the frameworks in Chapter 4 and from asking the IT team, my mentor, my teammates and AI tooling.

Security began after development. A teammate wrote the application code first, because I was new to this area and the access rules were not ready, and the team could not wait. That was the right call for a six-month phase, but it set the method: I could not build access control from a written table, because the code already existed. I read the code, wrote down what it allowed each role to do, compared that against the workflow the team needed, and changed the code where they differed. A substantial number of rules did not match what the hospital wanted, and some expected actions did not exist at all.

The domain was entirely new. Electronic medical records, the hospital’s own information system, the FHIR standard, and how insurance approval works were all unfamiliar. Before any of us could start building, we spent weeks understanding the process itself, what the hospital already had, what they wanted, and how people actually worked. Access control cannot be designed without understanding the workflow it protects, so learning the domain was a prerequisite rather than background reading.

Evaluation used the time that implementation would have needed. The team spent several months trialling Microsoft Entra ID before it was ruled out. When the hospital’s IT team was consulted about the firewall rules Entra ID would need, it became clear the hospital did not want the system connected to the cloud at all. Keycloak was then deployed and learned from the beginning. Federation with the hospital’s staff directory was delayed by the time spent on the Entra ID trial. It is now complete, and Section 5.1 describes the result.

Almost everything was designed by revision. Access rules were built one at a time, decided, implemented, checked, and sometimes the requirement itself changed once people saw the result. The audit trail had no template at all: I decided what to record, revised it, took it to the IT head, revised again, researched, and revised once more. Verification behaved the same way, since each time I finished checking, a teammate found something that sent the work back round. The gaps recorded in Chapter 6 reflect a design arrived at by iteration rather than one worked out in advance.

7.1.2 Stakeholder Challenges

The workflow had to be understood before it could be secured. Who may do what depends on which step a case has reached, so the flow had to be settled before the rules enforcing it could be written. The flow follows the hospital’s existing paper procedure, so I began with the paper, reading the forms, following how a file moved between departments, and learning the terms doctors and hospital staff use for what those forms contain. None of that vocabulary was familiar. The flow was not mine to settle either: the Coordinator and Finance teams had to agree to it between them, over a series of meetings, and each change to it changed the rules enforcing it.

Even now, with the flow decided and built, it is not certain that it matches what those teams expect in daily use.

Some findings were questions rather than defects. Whether the FHIR server is needed at all is a decision for the hospital, not for me. Whether the Coordinator should keep any ability to change prices after Finance has confirmed them was also such a question; it was raised with the hospital's team and settled during this work, in favour of removing that ability rather than keeping it, described in Section 5.1. Questions of this kind took longer to settle than code fixes. Changes of that kind also affected my teammates, locking a form or restricting an action alters what they are building against, so they were raised before being applied rather than afterwards.

7.2 Conclusion

This project secured a system that automates the Guarantee of Payment process at a Cambodian hospital. An identity provider was deployed on the hospital's own servers, access control was built across forty-two defined actions for five roles, an append-only audit trail was added, and the result was checked by two methods before deployment.

The clearest result came from comparing those methods. Static analysis applied 257 rules across 356 files and returned fifteen findings, none about access control. Role-by-role testing of the same system in the same period produced forty-seven findings, twenty-three of them access control. The scanner was not badly configured: access rules here come from how the hospital divides responsibility, not from anything visible in the shape of the code, and a tool cannot check a rule it was never given.

The second result concerns what those rules depend on. A permission is decided by three things together, the user's role, whether the case is theirs, and how far the case has progressed. The third is easiest to miss, because it changes the answer for a user whose role and assignment have not changed: a surgeon becomes read-only when the anaesthetist finishes and the case moves to Finance. That step is what enforces in software the separation the hospital already practised on paper.

The limits are stated rather than implied. There is no automated test suite, so verification was manual and happened once. The system was served without encryption at

the time of assessment, and two of its internal connections still are. Everything reported in this Graduation Project Report is self-assessment, by the person who built the controls; an independent penetration test has since been carried out by Canadia Bank (IT security department) and the hospital's IT Security Division, but its findings are confidential and not reproduced in this Graduation Project Report, so this Graduation Project Report cannot say whether it confirmed or contradicted what is reported here. And the final review found controls that had never run at all: a page-rule file the framework never loaded, and three background jobs nothing on the server started. Each has since taken a different path, superseded, given a working trigger, or retired outright, described in Section 5.4, but none of the three were fixed by the simple means first planned. That combination is the plainest evidence that a system checked once is not a system that stays checked.

What this project produced is therefore not a secure system but a documented one: rules that can be checked, a record of what was found, and a list of what remains with an owner against each item.

7.3 Future Work

Eighteen items remain. Some are not mine to decide. Setting the audit retention period, or deciding whether the disk should be encrypted, are choices for the hospital. Some need another team, such as the server administrator or Canadia Bank (IT security department), since they sit outside the application itself. The rest were within reach but ran out of time, and are left for the next phase.

Table 7.1: Work remaining, by priority.

	Item	Category	Priority	Who can carry it
1	Encrypt the Keycloak-to-application and Keycloak-to-AD connections	Server config	High	Server administrator

Continued on next page

Table 7.1 ... continued

	Item	Category	Priority	Who can carry it
2	Move Keycloak out of development mode in the production deployment profile, which its own documentation advises against	Server config	High	Server administrator
3	Add multi-factor authentication	Development	High	Security
4	Close System Administrator's write access to clinical and cost data on two routes, since the design states this role should be view-only there; the audit log records these writes accurately but with no flag marking them as out-of-policy, unlike the system's other admin override actions, which are deliberately logged to stand out	Development	High	Security
5	Hide raw error messages from users in production (ASVS F4)	Development	Medium	Security
6	Ask for password again before finalising a case or sending it to the insurer (ASVS F6)	Development	Medium	Security

Continued on next page

Table 7.1 ...continued

	Item	Category	Priority	Who can carry it
7	Send an alert when someone fails to log in or access something too many times	Development	Medium	Security
8	Record device/network address in audit entries	Development	Medium	Security
9	Add a way to force-log-out one specific session, without disabling the whole account	Development	Medium	Security
10	Check if the server can handle heavy use and stay running (DoS / availability)	Server config	Medium	Canada Bank (IT security department) (reviewed), Hospital IT
11	Add automated dependency and code scanning to the build pipeline, so a fixed vulnerability does not silently reappear in a later release without anyone noticing	Server config	Medium	Security
12	Write down when someone is blocked for having no role, since this is not recorded anywhere yet	Design	Medium	Security
13	Combine the case-assignment rule into one place (8 separate copies currently exist)	Design	Medium	Security

Continued on next page

Table 7.1 ... continued

	Item	Category	Priority	Who can carry it
14	Re-assess the system against a newer version of ASVS (5.0), since 4.0.3 and OWASP Top 10:2021 are both now several years old	Assessment	Medium	Security
15	Confirm whether Canadia Bank's container review specifically followed the CIS Docker Benchmark, or complete that comparison separately if not	Assessment	Low	Canadia Bank (IT security department) (reviewed), Hospital IT
16	Make the Actor column match the Description text after someone renames	Design	Low	Security
17	Give the same "not allowed" message whether a case is real or fake	Design	Low	Security
18	Fully remove the developer sign-in shortcut's code, rather than leaving it present and flag-gated, so it cannot be reintroduced by a future configuration change	Development	Medium	Security

References

- [1] Intercare Hospital, *Intercare hospital*, <https://intercarehospital.com/>, Accessed: Jul. 28, 2026, 2026.
- [2] U. N. Cobrado, S. Sharief, N. G. Regahal, E. Zepka, M. Mamauag, and L. C. Velasco, “Access control solutions in electronic health record systems: A systematic review,” *Informatics in Medicine Unlocked*, vol. 49, 2024, Accessed: Jul. 29, 2026.
- [3] HL7 International, *Security*, HL7 FHIR Release 5. <https://hl7.org/fhir/security.html>, Accessed: Jul. 28, 2026, 2026.
- [4] V. C. Hu et al., “Guide to attribute based access control (abac) definition and considerations,” National Institute of Standards and Technology, Tech. Rep. NIST Special Publication 800-162, Jan. 2014, Accessed: Jul. 28, 2026.
- [5] H. F. Atlam and Y. Yang, “Enhancing healthcare security: A unified rbac and abac risk-aware access control approach,” *Future Internet*, vol. 17, no. 6, p. 262, Jun. 2025. DOI: 10.3390/fi17060262
- [6] National Bank of Cambodia, “Technology and cyber risk management guidelines,” Tech. Rep., Jan. 2026, Accessed: Jul. 28, 2026.
- [7] National Institute of Standards and Technology, “The nist cybersecurity framework (csf) 2.0,” Tech. Rep. NIST Cybersecurity White Paper NIST CSWP 29, Feb. 2024, Accessed: Jul. 28, 2026.
- [8] Open Worldwide Application Security Project, *Owasp top 10:2021*, <https://owasp.org/Top10/2021/>, Accessed: Jul. 28, 2026, 2021.

- [9] Open Worldwide Application Security Project, *Owasp application security verification standard (asvs)*, <https://owasp.org/www-project-application-security-verification-standard/>, Accessed: Sep. 13, 2026, 2021.
- [10] Open Worldwide Application Security Project, *Owasp top 10:2025*, <https://owasp.org/Top10/2025/>, Accessed: Jul. 28, 2026, 2025.
- [11] U.S. Department of Health and Human Services, *The security rule*, <https://www.hhs.gov/hipaa/for-professionals/security/index.html>, Accessed: Sep. 15, 2026, 2024.
- [12] International Organization for Standardization, *Iso/iec 27001:2022, information security management systems*, <https://www.iso.org/standard/27001>, Accessed: Sep. 15, 2026, 2022.
- [13] Microsoft, *The stride threat model*, Microsoft Learn. [https://learn.microsoft.com/en-us/previous-versions/commerce-server/ee823878\(v=cs.20\)](https://learn.microsoft.com/en-us/previous-versions/commerce-server/ee823878(v=cs.20)), Accessed: Jul. 2026, 2026.
- [14] F. Sun, L. Xu, and Z. Su, “Static detection of access control vulnerabilities in web applications,” in *Proc. 20th USENIX Security Symposium*, Accessed: Jul. 29, 2026, San Francisco, CA, 2011.
- [15] J. Zhu, B. Chu, H. Lipford, and T. Thomas, “Mitigating access control vulnerabilities through interactive static analysis,” in *Proc. 20th ACM Symposium on Access Control Models and Technologies*, Jun. 2015. DOI: 10.1145/2752952.2752976
- [16] A. Anas, S. Elgamal, and B. Youssef, “Survey on detecting and preventing web application broken access control attacks,” *International Journal of Electrical and Computer Engineering (IJECE)*, vol. 14, no. 1, pp. 772–781, Feb. 2024. DOI: 10.11591/ijece.v14i1.pp772-781